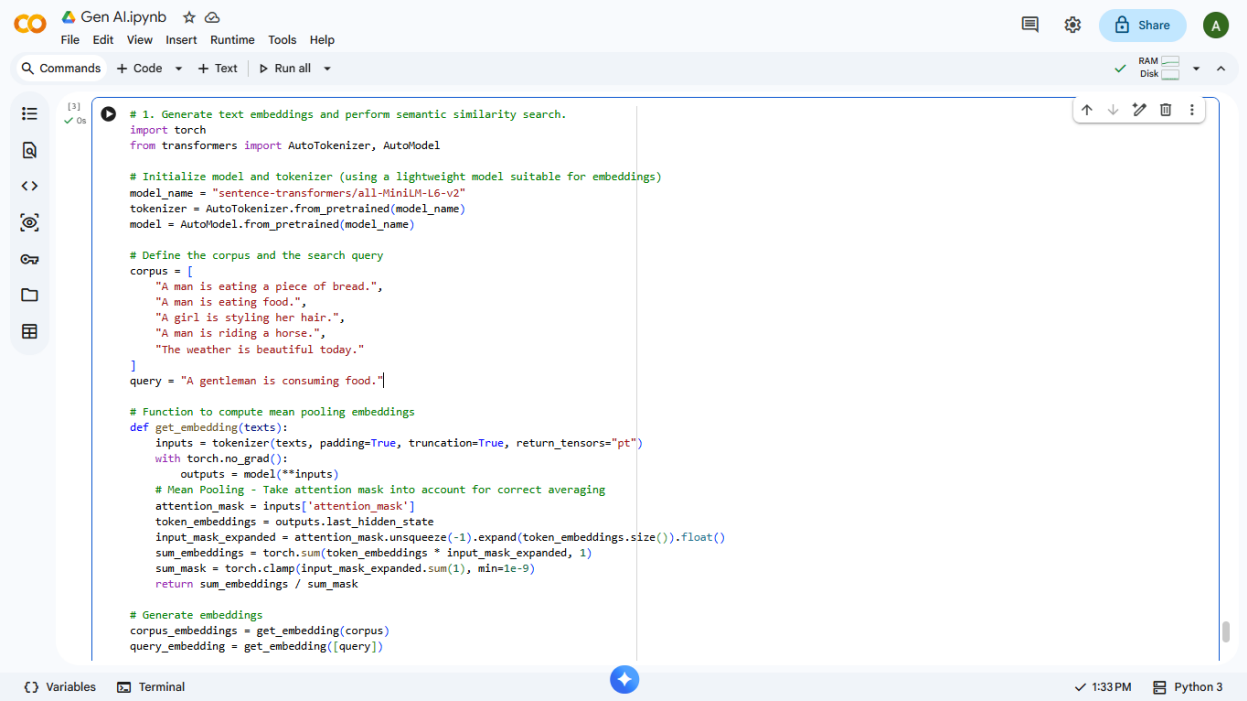
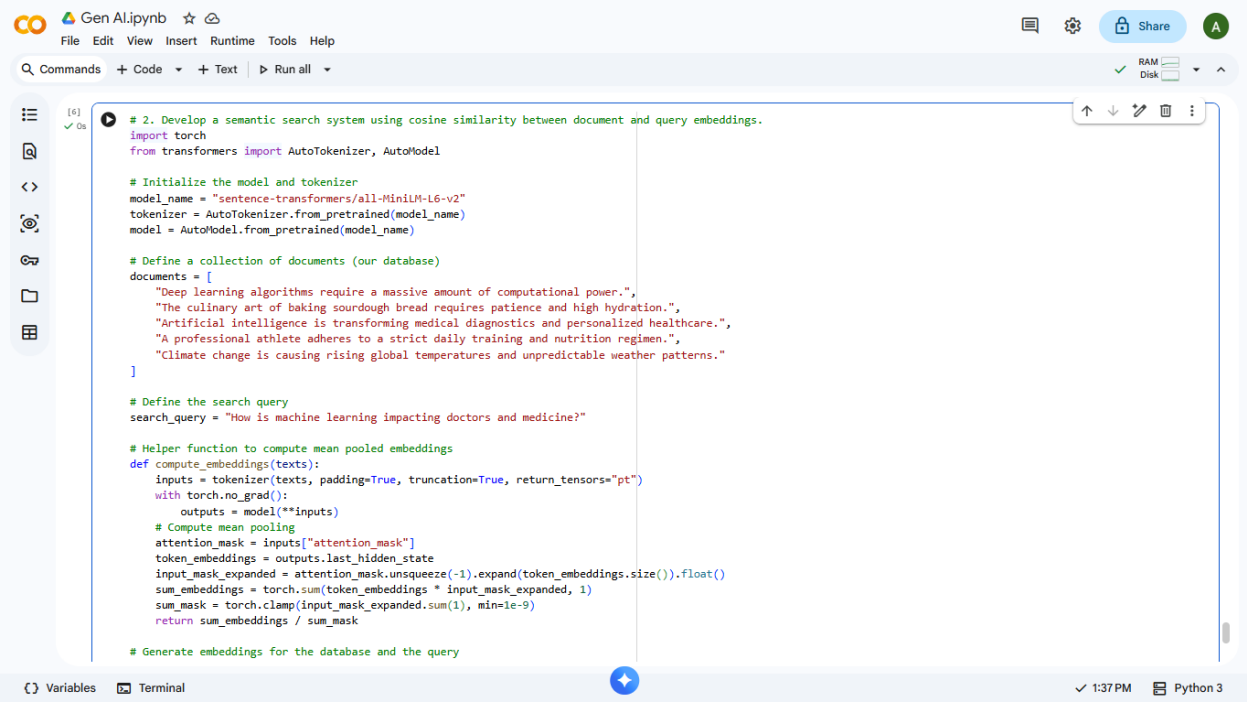
1. Generate text embeddings and perform semantic similarity search.



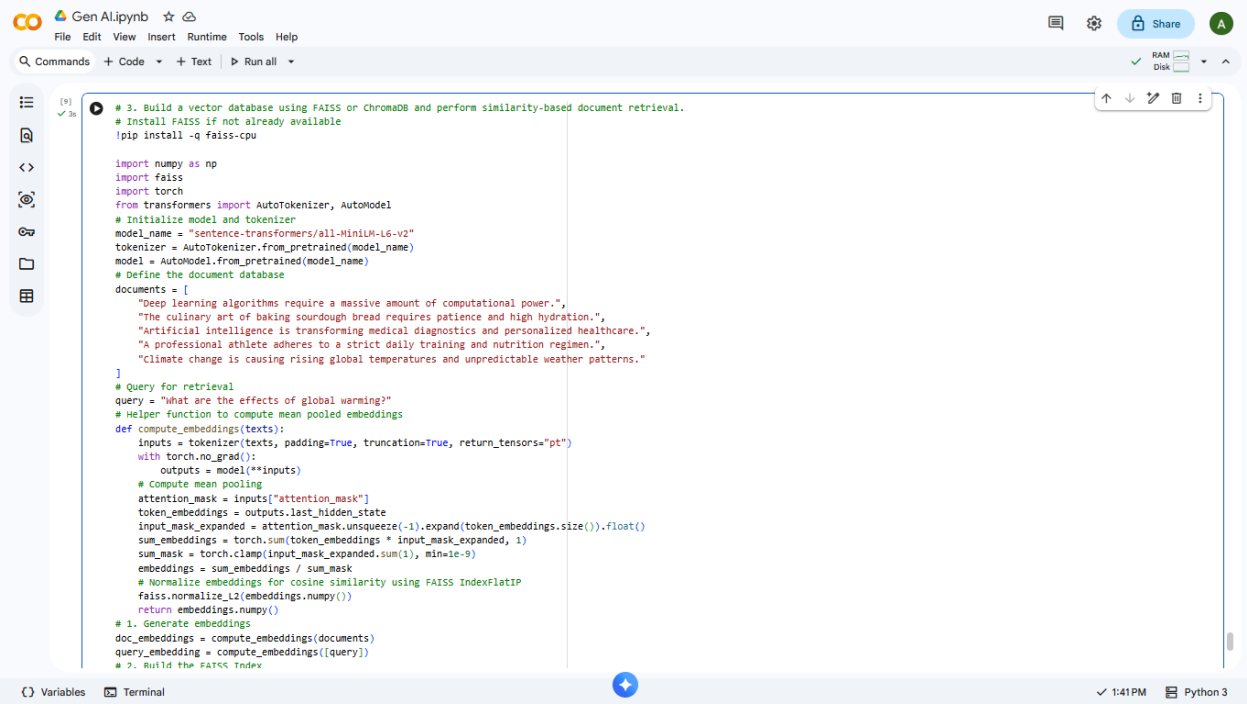


2. Develop a semantic search system using cosine similarity between document and query embeddings.



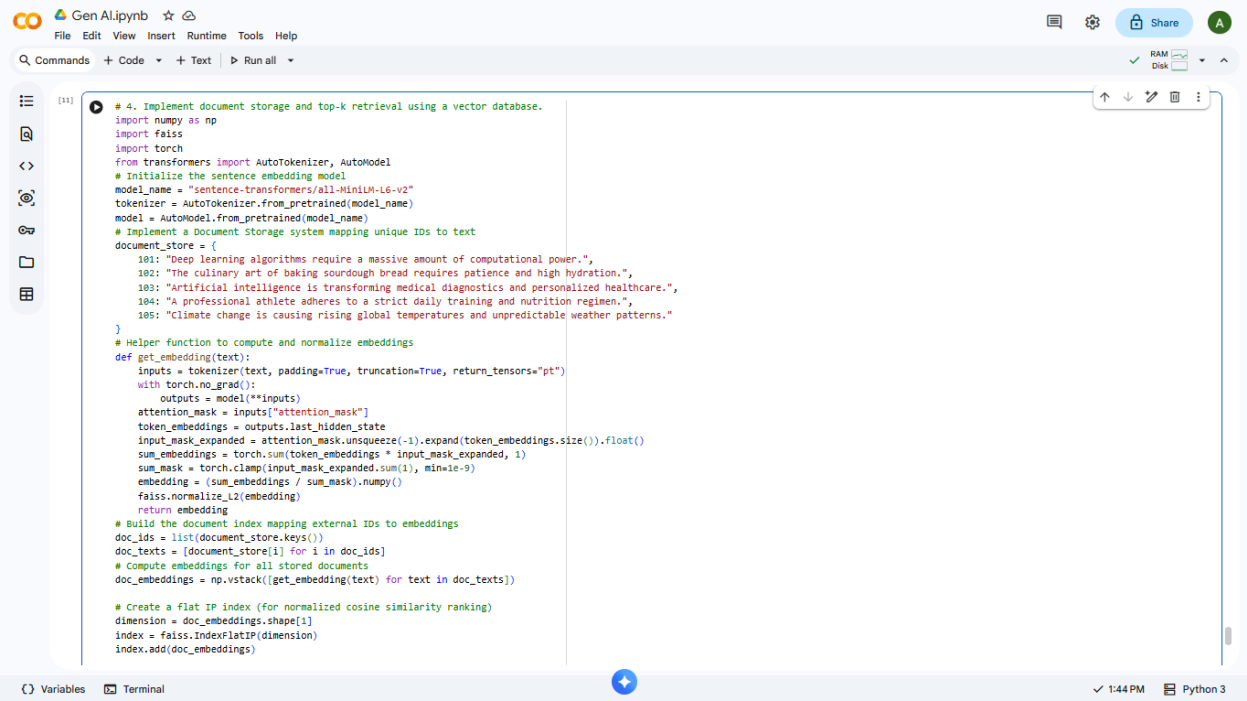


3. Build a vector database using FAISS or ChromaDB and perform similarity based document retrieval.



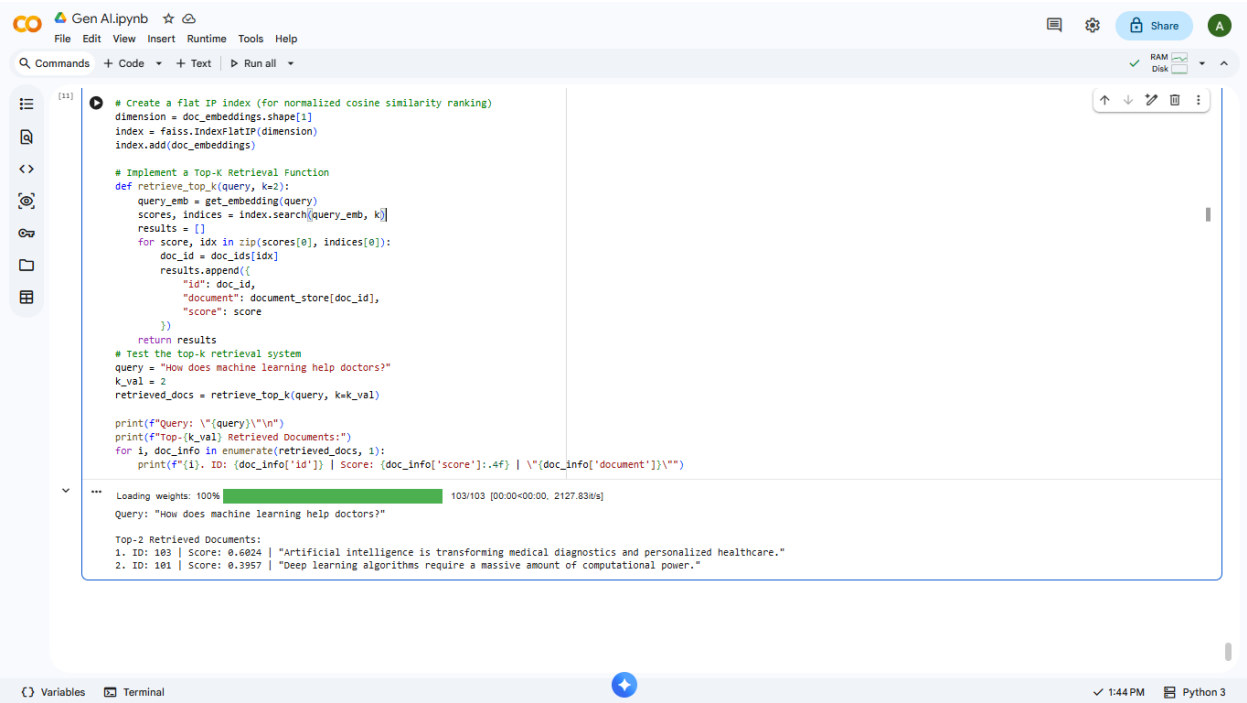


4. Implement document storage and top-k retrieval using a vector database.



```
# 4. Implement document storage and top-k retrieval using a vector database.
import numpy as np
import faiss
import torch
from transformers import AutoTokenizer, AutoModel
# Initialize the sentence embedding model
model_name = "sentence-transformers/all-MiniLM-L6-v2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
# Implement a Document Storage system mapping unique IDs to text
document_store = {
    101: "Deep learning algorithms require a massive amount of computational power.",
    102: "The culinary art of baking sourdough bread requires patience and high hydration.",
    103: "Artificial intelligence is transforming medical diagnostics and personalized healthcare.",
    104: "A professional athlete adheres to a strict daily training and nutrition regimen.",
    105: "Climate change is causing rising global temperatures and unpredictable weather patterns."
}
# Helper function to compute and normalize embeddings
def get_embedding(text):
    inputs = tokenizer(text, padding=True, truncation=True, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)
    attention_mask = inputs["attention_mask"]
    token_embeddings = outputs.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    sum_embeddings = torch.sum(token_embeddings * input_mask_expanded, 1)
    sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
    embedding = (sum_embeddings / sum_mask).numpy()
    faiss.normalize_L2(embedding)
    return embedding
# Build the document index mapping external IDs to embeddings
doc_ids = list(document_store.keys())
doc_texts = [document_store[i] for i in doc_ids]
# Compute embeddings for all stored documents
doc_embeddings = np.vstack([get_embedding(text) for text in doc_texts])

# Create a flat IP index (for normalized cosine similarity ranking)
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(doc_embeddings)
```



```
# Create a flat IP index (for normalized cosine similarity ranking)
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(doc_embeddings)

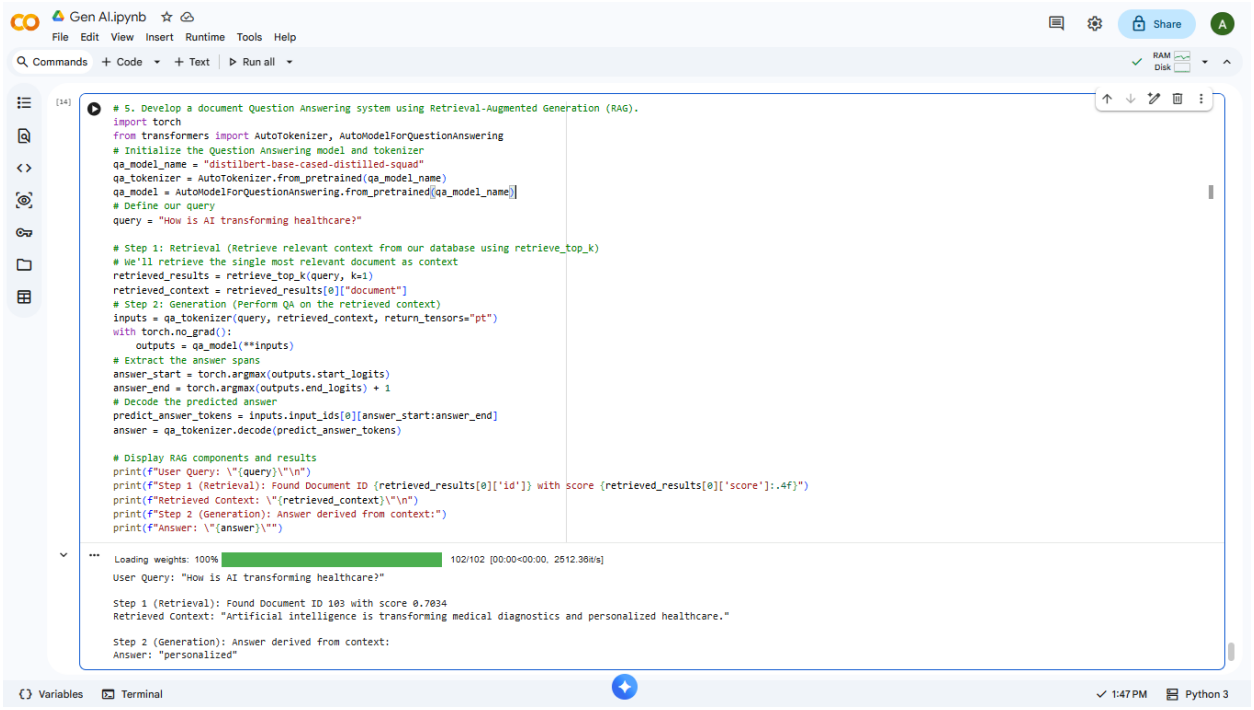
# Implement a Top-K Retrieval Function
def retrieve_top_k(query, k=2):
    query_emb = get_embedding(query)
    scores, indices = index.search(query_emb, k)
    results = []
    for score, idx in zip(scores[0], indices[0]):
        doc_id = doc_ids[idx]
        results.append({
            "id": doc_id,
            "document": document_store[doc_id],
            "score": score
        })
    return results
# Test the top-k retrieval system
query = "How does machine learning help doctors?"
k_val = 2
retrieved_docs = retrieve_top_k(query, k=k_val)

print(f"Query: \"{query}\"\n")
print(f"Top-{k_val} Retrieved Documents:")
for i, doc_info in enumerate(retrieved_docs, 1):
    print(f"{i}. ID: {doc_info['id']} | Score: {doc_info['score']:.4f} | \"{doc_info['document']}\"")
```

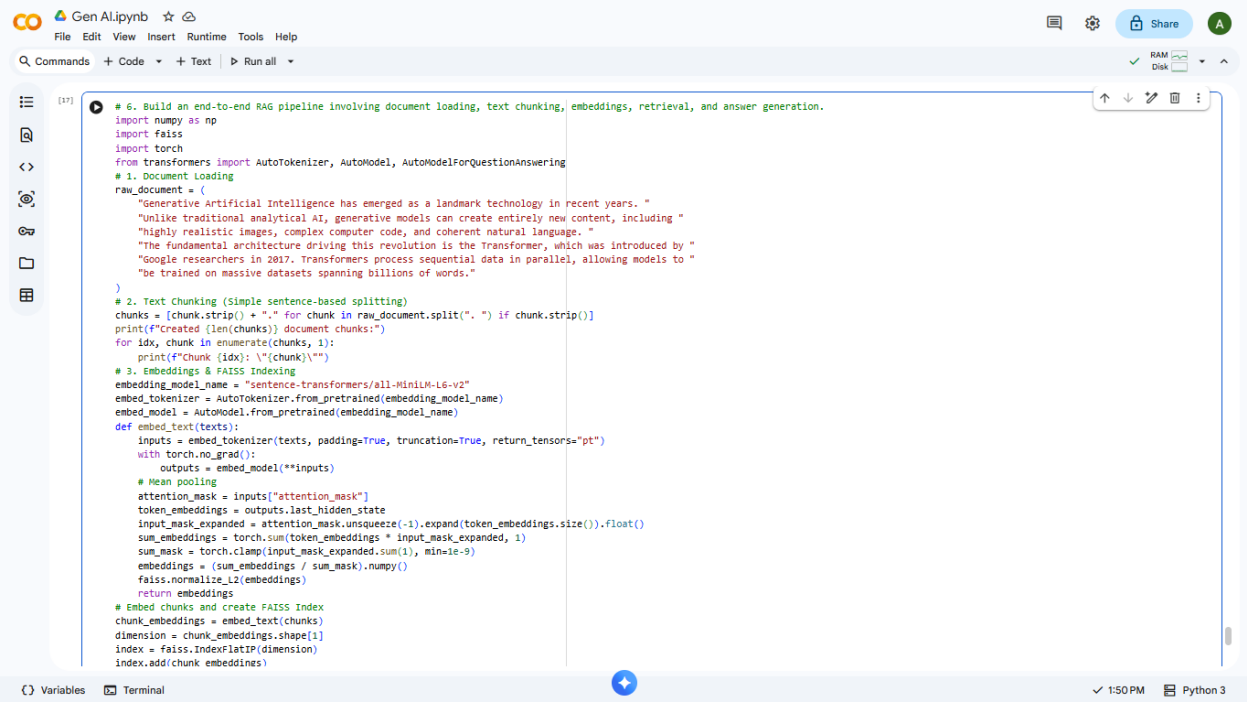
```
Loading weights: 100% 103/103 [00:00<00:00, 2127.83it/s]
Query: "How does machine learning help doctors?"

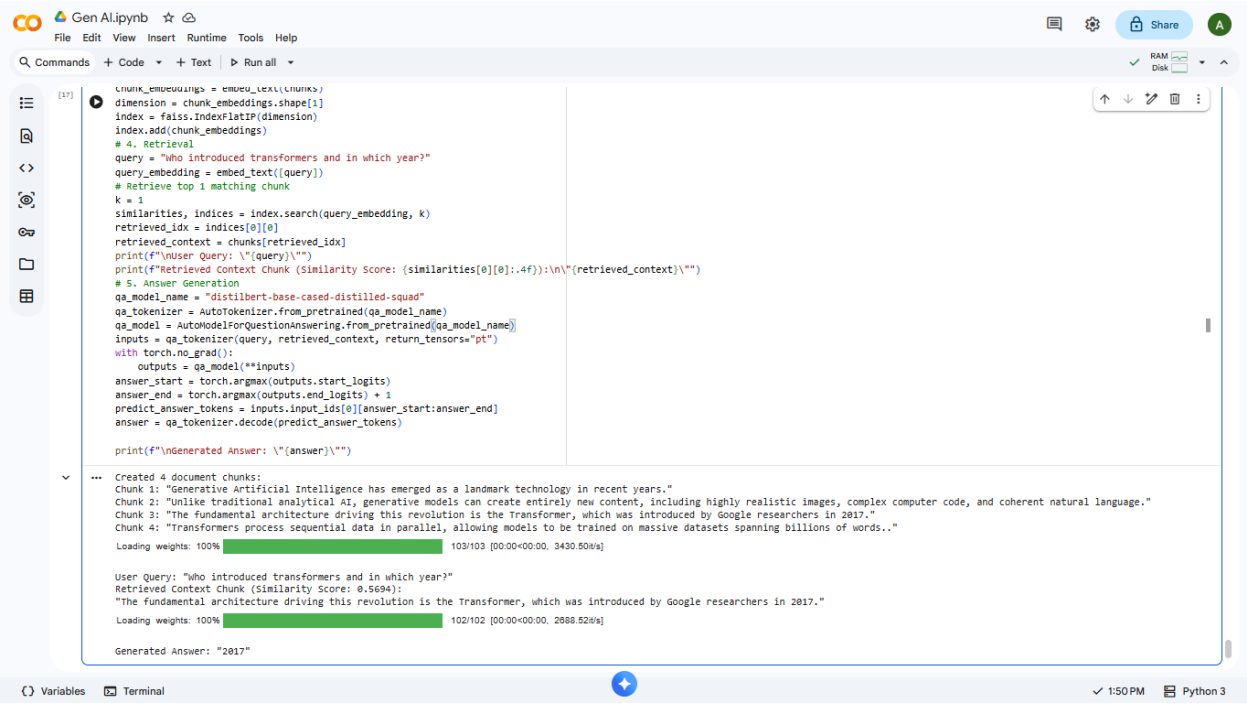
Top-2 Retrieved Documents:
1. ID: 103 | Score: 0.6024 | "Artificial intelligence is transforming medical diagnostics and personalized healthcare."
2. ID: 101 | Score: 0.3957 | "Deep learning algorithms require a massive amount of computational power."
```

5. Develop a document Question Answering system using Retrieval- Augmented Generation (RAG).

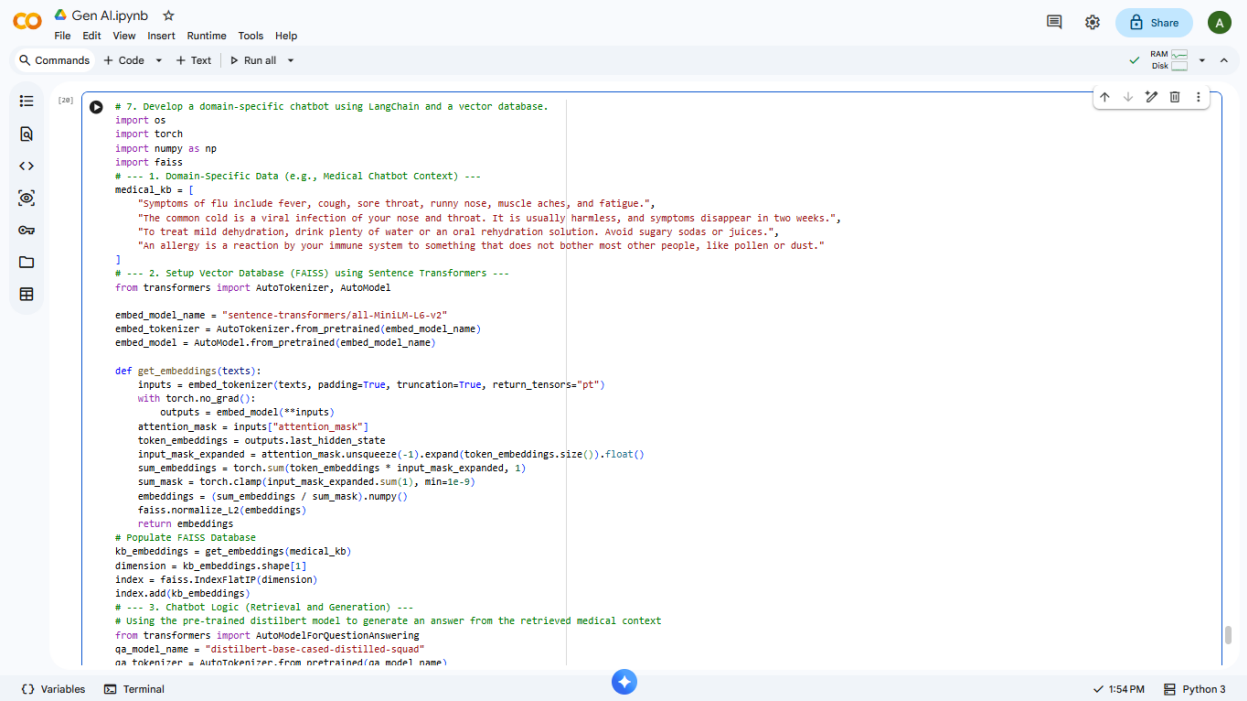


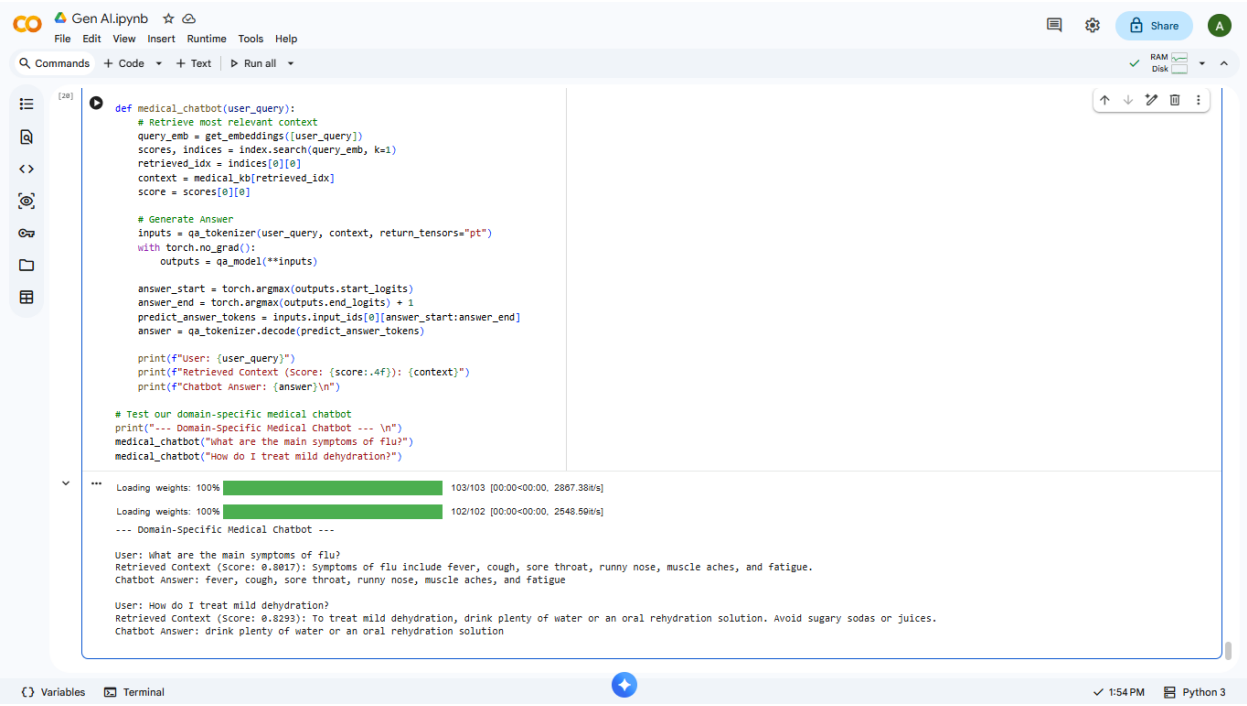
6. Build an end-to-end RAG pipeline involving document loading, text chunking, embeddings, retrieval, and answer generation.



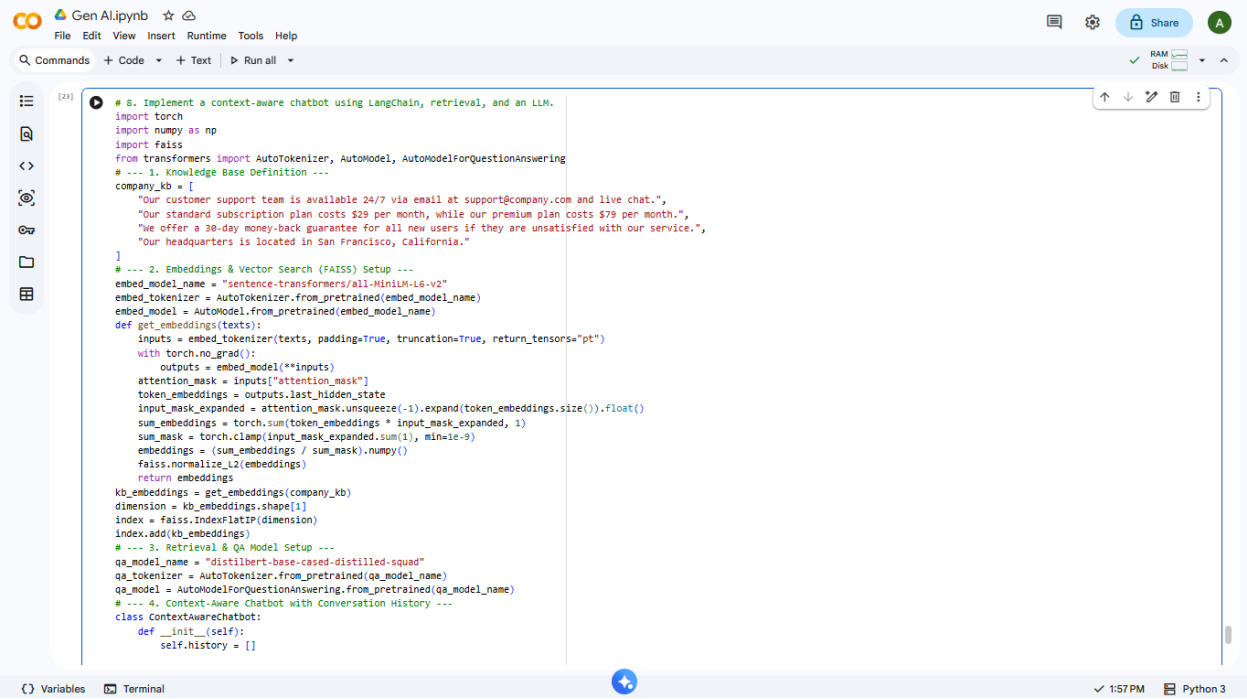


7. Develop a domain-specific chatbot using Lang Chain and a vector database.

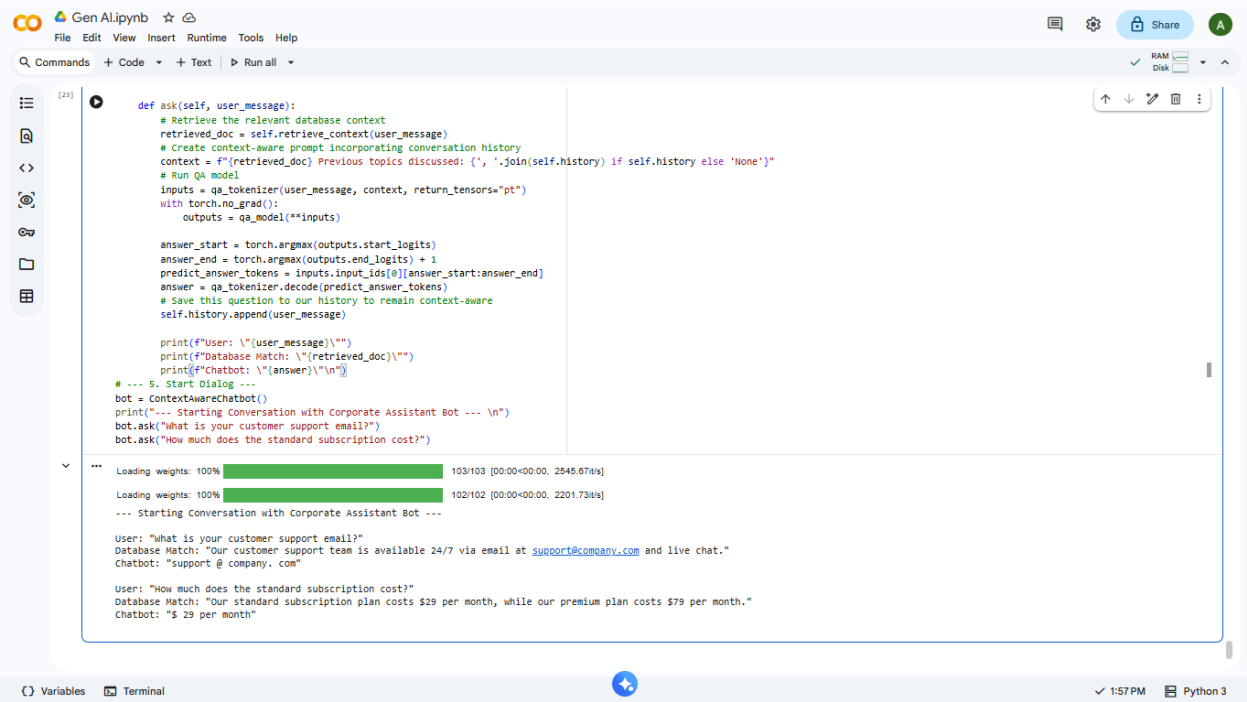




8. Implement a context-aware chatbot using Lang Chain, retrieval, and an LLM.



```
# 8. Implement a context-aware chatbot using LangChain, retrieval, and an LLM.
import torch
import numpy as np
import faiss
from transformers import AutoTokenizer, AutoModel, AutoModelForQuestionAnswering
# --- 1. Knowledge Base Definition ---
company_kb = [
    "Our customer support team is available 24/7 via email at support@company.com and live chat.",
    "Our standard subscription plan costs $29 per month, while our premium plan costs $79 per month.",
    "We offer a 30-day money-back guarantee for all new users if they are unsatisfied with our service.",
    "Our headquarters is located in San Francisco, California."
]
# --- 2. Embeddings & Vector Search (FAISS) Setup ---
embed_model_name = "sentence-transformers/all-MiniLM-L6-v2"
embed_tokenizer = AutoTokenizer.from_pretrained(embed_model_name)
embed_model = AutoModel.from_pretrained(embed_model_name)
def get_embeddings(texts):
    inputs = embed_tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    with torch.no_grad():
        outputs = embed_model(**inputs)
    attention_mask = inputs["attention_mask"]
    token_embeddings = outputs.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    sum_embeddings = torch.sum(token_embeddings * input_mask_expanded, 1)
    sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
    embeddings = (sum_embeddings / sum_mask).numpy()
    faiss.normalize_L2(embeddings)
    return embeddings
kb_embeddings = get_embeddings(company_kb)
dimension = kb_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(kb_embeddings)
# --- 3. Retrieval & QA Model Setup ---
qa_model_name = "distilbert-base-cased-distilled-squad"
qa_tokenizer = AutoTokenizer.from_pretrained(qa_model_name)
qa_model = AutoModelForQuestionAnswering.from_pretrained(qa_model_name)
# --- 4. Context-Aware Chatbot with Conversation History ---
class ContextAwareChatbot:
    def __init__(self):
        self.history = []
```



```
    def ask(self, user_message):
        # Retrieve the relevant database context
        retrieved_doc = self.retrieve_context(user_message)
        # Create context-aware prompt incorporating conversation history
        context = f"{retrieved_doc} Previous topics discussed: {', '.join(self.history) if self.history else 'None'}"
        # Run QA model
        inputs = qa_tokenizer(user_message, context, return_tensors="pt")
        with torch.no_grad():
            outputs = qa_model(**inputs)

        answer_start = torch.argmax(outputs.start_logits)
        answer_end = torch.argmax(outputs.end_logits) + 1
        predict_answer_tokens = inputs.input_ids[0][answer_start:answer_end]
        answer = qa_tokenizer.decode(predict_answer_tokens)
        # Save this question to our history to remain context-aware
        self.history.append(user_message)

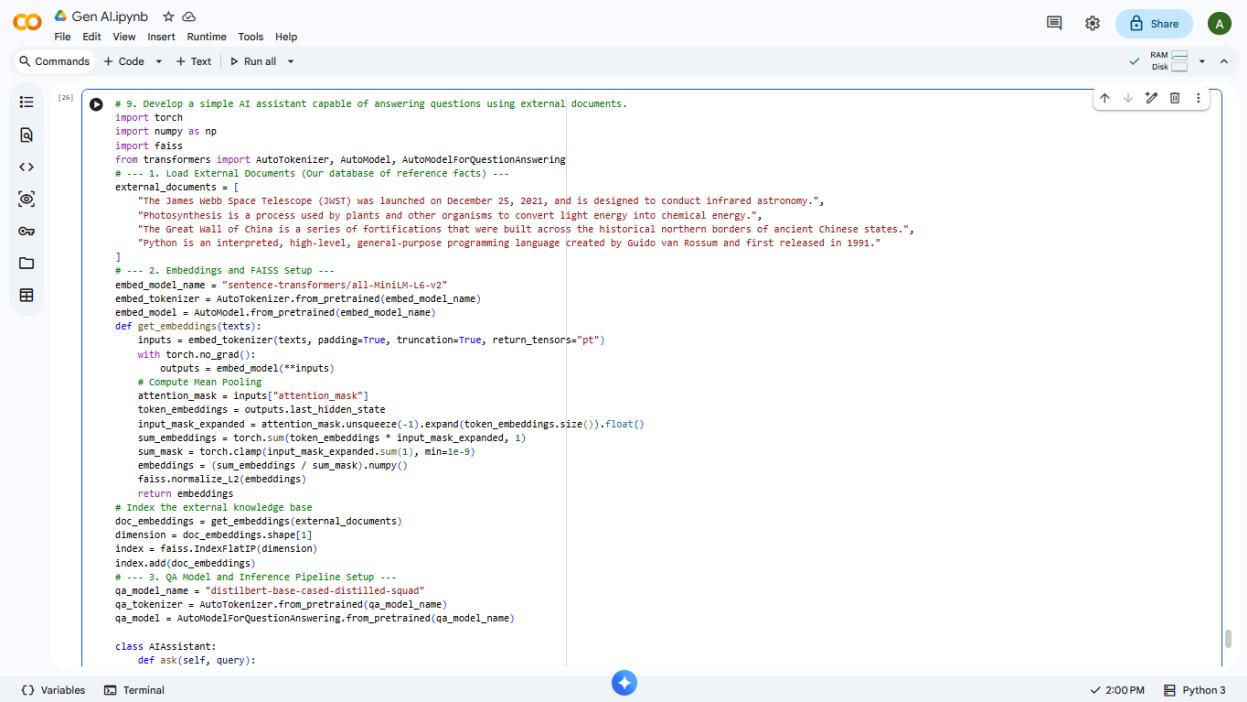
        print(f"User: \"{user_message}\"")
        print(f"Database Match: \"{retrieved_doc}\"")
        print(f"Chatbot: \"{answer}\"\n")
# --- 5. Start Dialog ---
bot = ContextAwareChatbot()
print("--- Starting Conversation with Corporate Assistant Bot --- \n")
bot.ask("What is your customer support email?")
bot.ask("How much does the standard subscription cost?")
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 2545.67it/s]
Loading weights: 100%|██████████| 102/102 [00:00<00:00, 2201.73it/s]
--- Starting Conversation with Corporate Assistant Bot ---

User: "What is your customer support email?"
Database Match: "Our customer support team is available 24/7 via email at support@company.com and live chat."
Chatbot: "support @ company. com"

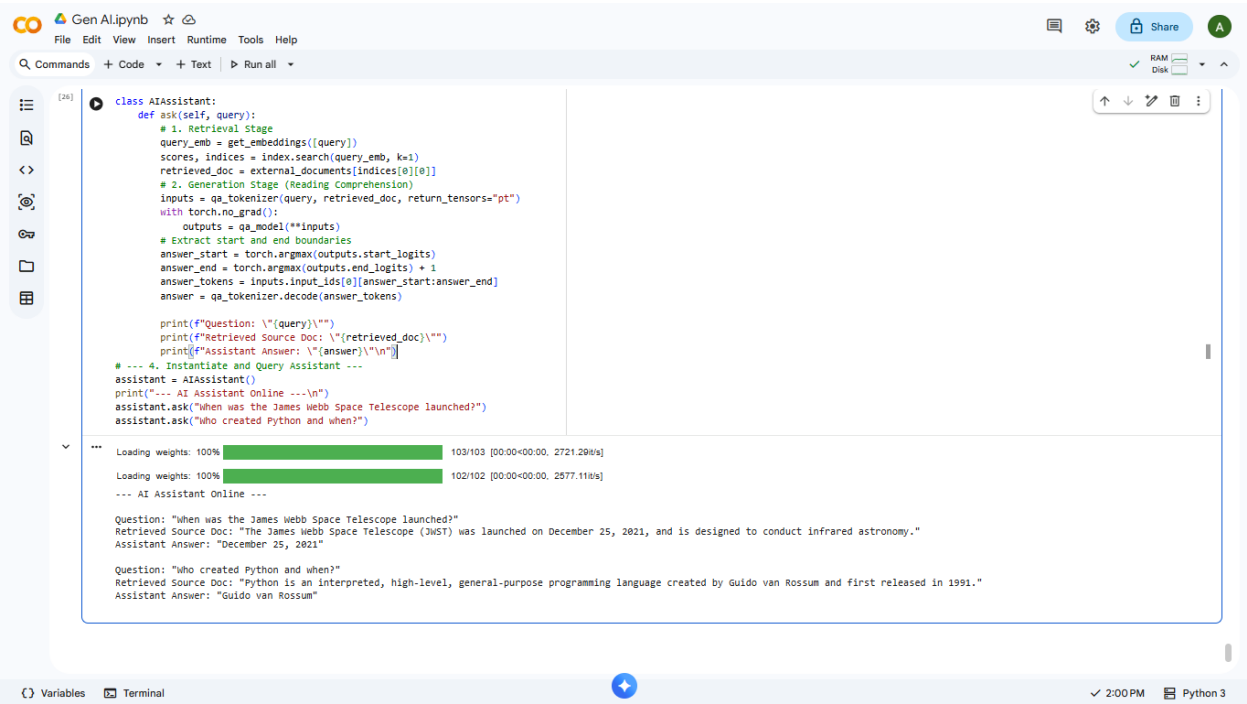
User: "How much does the standard subscription cost?"
Database Match: "Our standard subscription plan costs $29 per month, while our premium plan costs $79 per month."
Chatbot: "$ 29 per month"
```

9. Develop a simple AI assistant capable of answering questions using external documents.



```
# 9. Develop a simple AI assistant capable of answering questions using external documents.
import torch
import numpy as np
import faiss
from transformers import AutoTokenizer, AutoModel, AutoModelForQuestionAnswering
# --- 1. Load External Documents (Our database of reference facts) ---
external_documents = [
    "The James Webb Space Telescope (JWST) was launched on December 25, 2021, and is designed to conduct infrared astronomy.",
    "Photosynthesis is a process used by plants and other organisms to convert light energy into chemical energy.",
    "The Great Wall of China is a series of fortifications that were built across the historical northern borders of ancient Chinese states.",
    "Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991."
]
# --- 2. Embeddings and FAISS Setup ---
embed_model_name = "sentence-transformers/all-MiniLM-L6-v2"
embed_tokenizer = AutoTokenizer.from_pretrained(embed_model_name)
embed_model = AutoModel.from_pretrained(embed_model_name)
def get_embeddings(texts):
    inputs = embed_tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    with torch.no_grad():
        outputs = embed_model(**inputs)
    # Compute Mean Pooling
    attention_mask = inputs["attention_mask"]
    token_embeddings = outputs.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    sum_embeddings = torch.sum(token_embeddings * input_mask_expanded, 1)
    sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
    embeddings = (sum_embeddings / sum_mask).numpy()
    faiss.normalize_L2(embeddings)
    return embeddings
# Index the external knowledge base
doc_embeddings = get_embeddings(external_documents)
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(doc_embeddings)
# --- 3. QA Model and Inference Pipeline Setup ---
qa_model_name = "distilbert-base-cased-distilled-squad"
qa_tokenizer = AutoTokenizer.from_pretrained(qa_model_name)
qa_model = AutoModelForQuestionAnswering.from_pretrained(qa_model_name)

class AIAssistant:
    def ask(self, query):
```



```
class AIAssistant:
    def ask(self, query):
        # 1. Retrieval Stage
        query_emb = get_embeddings([query])
        scores, indices = index.search(query_emb, k=1)
        retrieved_doc = external_documents[indices[0][0]]
        # 2. Generation Stage (Reading Comprehension)
        inputs = qa_tokenizer(query, retrieved_doc, return_tensors="pt")
        with torch.no_grad():
            outputs = qa_model(**inputs)
        # Extract start and end boundaries
        answer_start = torch.argmax(outputs.start_logits)
        answer_end = torch.argmax(outputs.end_logits) + 1
        answer_tokens = inputs.input_ids[0][answer_start:answer_end]
        answer = qa_tokenizer.decode(answer_tokens)

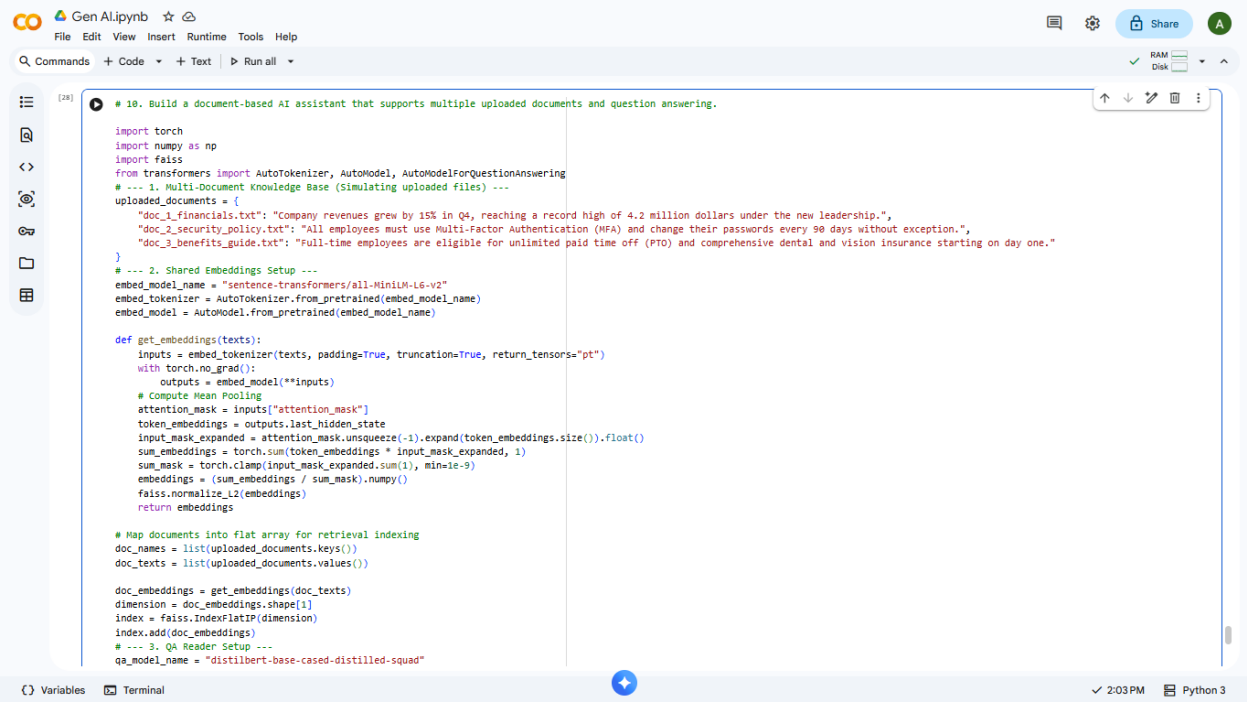
        print(f"Question: \"{query}\"")
        print(f"Retrieved Source Doc: \"{retrieved_doc}\"")
        print(f"Assistant Answer: \"{answer}\"\n")
# --- 4. Instantiate and Query Assistant ---
assistant = AIAssistant()
print("--- AI Assistant Online ---\n")
assistant.ask("When was the James Webb Space Telescope launched?")
assistant.ask("Who created Python and when?")
```

```
Loading weights: 100% 103/103 [00:00<00:00, 2721.29it/s]
Loading weights: 100% 102/102 [00:00<00:00, 2577.11it/s]
--- AI Assistant Online ---

Question: "When was the James Webb Space Telescope launched?"
Retrieved Source Doc: "The James Webb Space Telescope (JWST) was launched on December 25, 2021, and is designed to conduct infrared astronomy."
Assistant Answer: "December 25, 2021"

Question: "Who created Python and when?"
Retrieved Source Doc: "Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991."
Assistant Answer: "Guido van Rossum"
```

10. Build a document-based AI assistant that supports multiple uploaded documents and question answering.



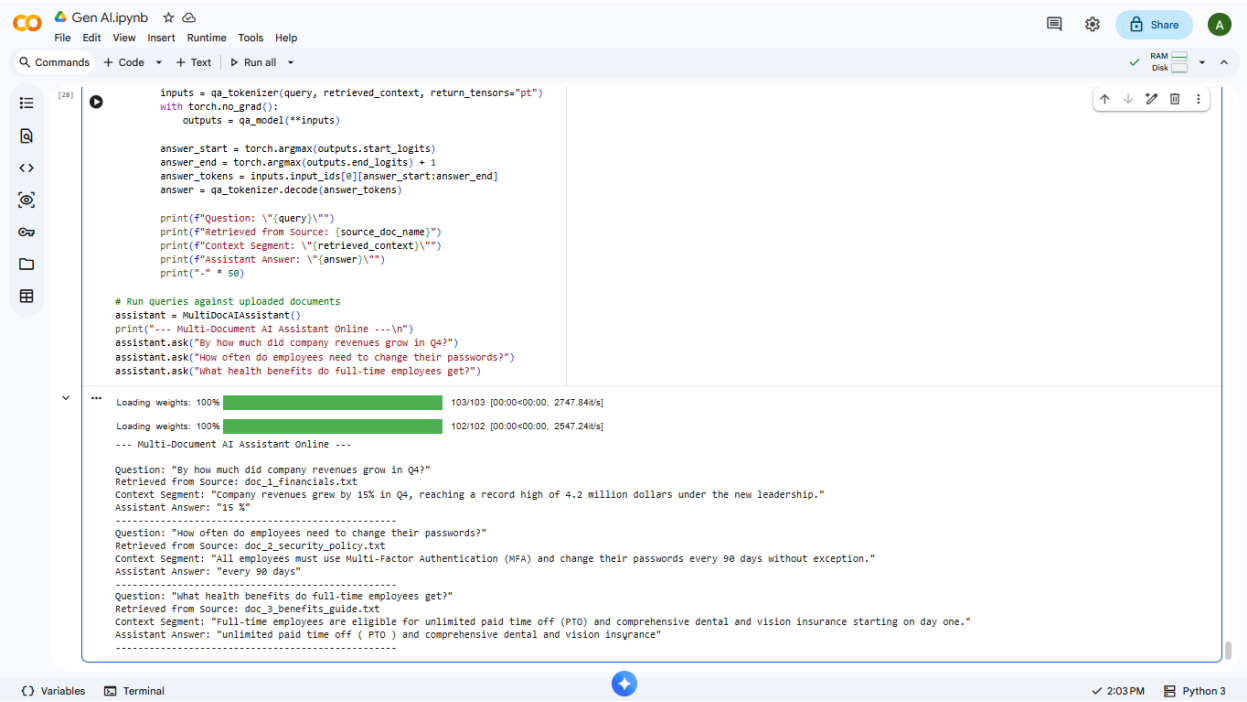
```
# 10. Build a document-based AI assistant that supports multiple uploaded documents and question answering.

import torch
import numpy as np
import faiss
from transformers import AutoTokenizer, AutoModel, AutoModelForQuestionAnswering
# --- 1. Multi-Document Knowledge Base (Simulating uploaded files) ---
uploaded_documents = {
    "doc_1_financials.txt": "Company revenues grew by 15% in Q4, reaching a record high of 4.2 million dollars under the new leadership.",
    "doc_2_security_policy.txt": "All employees must use Multi-Factor Authentication (MFA) and change their passwords every 90 days without exception.",
    "doc_3_benefits_guide.txt": "Full-time employees are eligible for unlimited paid time off (PTO) and comprehensive dental and vision insurance starting on day one."
}
# --- 2. Shared Embeddings Setup ---
embed_model_name = "sentence-transformers/all-MiniLM-L6-v2"
embed_tokenizer = AutoTokenizer.from_pretrained(embed_model_name)
embed_model = AutoModel.from_pretrained(embed_model_name)

def get_embeddings(texts):
    inputs = embed_tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    with torch.no_grad():
        outputs = embed_model(**inputs)
    # Compute Mean Pooling
    attention_mask = inputs["attention_mask"]
    token_embeddings = outputs.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    sum_embeddings = torch.sum(token_embeddings * input_mask_expanded, 1)
    sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
    embeddings = (sum_embeddings / sum_mask).numpy()
    faiss.normalize_L2(embeddings)
    return embeddings

# Map documents into flat array for retrieval indexing
doc_names = list(uploaded_documents.keys())
doc_texts = list(uploaded_documents.values())

doc_embeddings = get_embeddings(doc_texts)
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(doc_embeddings)
# --- 3. QA Reader Setup ---
qa_model_name = "distilbert-base-cased-distilled-squad"
```



```
        inputs = qa_tokenizer(query, retrieved_context, return_tensors="pt")
        with torch.no_grad():
            outputs = qa_model(**inputs)

        answer_start = torch.argmax(outputs.start_logits)
        answer_end = torch.argmax(outputs.end_logits) + 1
        answer_tokens = inputs.input_ids[0][answer_start:answer_end]
        answer = qa_tokenizer.decode(answer_tokens)

        print(f"Question: \"{query}\"")
        print(f"Retrieved from Source: {source_doc_name}")
        print(f"Context Segment: \"{retrieved_context}\"")
        print(f"Assistant Answer: \"{answer}\"")
        print("-" * 50)

# Run queries against uploaded documents
assistant = MultiDocAIAssistant()
print("--- Multi-Document AI Assistant Online ---\n")
assistant.ask("By how much did company revenues grow in Q4?")
assistant.ask("How often do employees need to change their passwords?")
assistant.ask("What health benefits do full-time employees get?")
```

```
Loading weights: 100% 103/103 [00:00<00:00, 2747.84it/s]
Loading weights: 100% 102/102 [00:00<00:00, 2547.24it/s]
--- Multi-Document AI Assistant Online ---

Question: "By how much did company revenues grow in Q4?"
Retrieved from Source: doc_1_financials.txt
Context Segment: "Company revenues grew by 15% in Q4, reaching a record high of 4.2 million dollars under the new leadership."
Assistant Answer: "15 %"
--------------------------------------------------
Question: "How often do employees need to change their passwords?"
Retrieved from Source: doc_2_security_policy.txt
Context Segment: "All employees must use Multi-Factor Authentication (MFA) and change their passwords every 90 days without exception."
Assistant Answer: "every 90 days"
--------------------------------------------------
Question: "What health benefits do full-time employees get?"
Retrieved from Source: doc_3_benefits_guide.txt
Context Segment: "Full-time employees are eligible for unlimited paid time off (PTO) and comprehensive dental and vision insurance starting on day one."
Assistant Answer: "unlimited paid time off ( PTO ) and comprehensive dental and vision insurance"
--------------------------------------------------
```